



Capitolo 8

Programmazione Logica: Il Linguaggio Prolog

8.1 Dalla Logica alla Programmazione Logica

- Come si è arrivati dalla logica alla programmazione logica?
La logica può essere definita come la disciplina che studia la verità di un'affermazione deducendola (tramite regole) da altre affermazioni che sono state poste essere vere (postulati, assiomi, principi) o che sono già state dedotte essere vere (teoremi). Dal punto di vista concettuale, la logica si contrappone all'intuizione e risponde all'esigenza di trovare un modo di distinguere ciò che è vero da ciò che è falso. La storia della logica si articola in tre epoche: logica simbolica (dal 500 a.C. alla metà del 1800), logica algebrica (seconda metà del 1800) e logica matematica (dalla fine del 1800 in poi).
- Nell'antica Grecia, la logica era già ritenuta una materia fondamentale. Essa venne originariamente studiata dai sofisti nel VI secolo a.C. allo scopo di definire un sistema oggettivo di regole per determinare oltre ogni dubbio chi avesse vinto una disputa. Nel IV secolo a.C. Aristotele diede alla logica la forma rigorosamente deduttiva del sillogismo, cioè del ragionamento concatenato attraverso cui si giunge a conclusioni coerenti con le premesse da cui si parte (tutte le persone sono mortali, Socrate è una persona, quindi Socrate è mortale). In ogni sillogismo, sia le premesse che la conclusione sono espressioni formate da un soggetto universale o particolare e da un predicato verbale affermativo o negativo, dove la verità della conclusione dipende dalla verità delle premesse. La logica di Aristotele si basa su due valori di verità ed è governata da tre leggi: il principio di identità (ogni oggetto è uguale a se stesso), il principio di non contraddizione (due espressioni in contraddizione non possono essere entrambe vere) e il principio del terzo escluso (*tertium non datur*: ogni espressione deve essere o vera o falsa).
- Inizialmente la logica trattava affermazioni espresse in linguaggio naturale e si basava su regole di ragionamento molto semplici. Sfortunatamente, l'intrinseca ambiguità del linguaggio naturale conduce facilmente a paradossi, spesso nella forma di autologie (affermazioni che fanno riferimento a se stesse, come il paradosso del bugiardo: "questa affermazione è una bugia").
- Nella seconda metà del 1600 Leibniz riteneva che una larga parte del ragionamento umano potesse essere ridotta a calcoli e che questi ultimi potessero essere usati per risolvere molte divergenze di opinioni (da cui il motto "calcelemus"). A tale scopo, Leibniz immaginò un linguaggio formale universale detto "characteristica universalis" in grado di esprimere concetti matematici, scientifici e metafisici, da usare nel contesto di un calcolo logico universale detto "calculus ratiocinator". Leibniz enunciò inoltre le principali proprietà di ciò che ora chiamiamo congiunzione, disgiunzione e negazione e introdusse il sistema di numerazione binario, i cui due simboli di base 0 e 1 possono essere messi in corrispondenza biunivoca con i due valori di verità falso e vero.

- La prima formulazione della logica in termini di un linguaggio matematico venne espressa nel 1847 da Boole nel suo libro “The Mathematical Analysis of Logic”. Il suo obiettivo era quello di investigare le leggi fondamentali delle operazioni che avvengono nella mente umana durante il ragionamento. Boole individuò un’analogia tra le operazioni aritmetiche di addizione e moltiplicazione e le operazioni insiemistiche di unione e intersezione: per esempio $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$ è simile ad $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$. Egli estese questa analogia alla logica introducendo e studiando le proprietà algebriche di un operatore di disgiunzione logica (OR) e un operatore di congiunzione logica (AND) accompagnati da un operatore di negazione logica (NOT) e dai valori di verità falso (0) e vero (1). Nello stesso anno De Morgan fornì nel suo libro “Formal Logic” degli importanti contributi allo studio dei sillogismi numericamente definiti. Nel 1880 Venn propose nel suo articolo “On the Diagrammatic and Mechanical Representation of Propositions and Reasonings” l’uso di particolari diagrammi per rappresentare gli insiemi e le loro operazioni che, grazie all’analogia di Boole, potevano essere impiegati anche nella logica. Alla fine del 1800 Schröder anticipò nel suo libro “The Algebra of Logic” l’importanza di trovare metodi rapidi per determinare le conseguenze di premesse arbitrarie.
- Nel XIX secolo ci si rese conto che, come la logica necessitava della matematica per superare l’ambiguità del linguaggio naturale, così la matematica necessitava di fondamenta adeguate da un lato e meccanismi deduttivi rigorosi per migliorare la qualità delle dimostrazioni dei teoremi dall’altro. Nel 1879 Frege propose nel suo “Begriffsschrift” la logica come un linguaggio per la matematica funzionale alla dimostrazione dei teoremi, dove eventuali elementi intuitivi o primitivi devono essere isolati come assiomi (o postulati o principi) e la prova deve procedere a partire da questi assiomi in modo puramente logico e senza salti. Frege introdusse inoltre un sistema logico con negazione, implicazione, quantificazione universale e tabelle di verità, che può essere visto come il primo calcolo dei predicati.
- Anche la formulazione delle teorie matematiche necessitava della logica. Per esempio, la cosiddetta teoria ingenua degli insiemi formulata da Cantor contiene numerosi paradossi: denotata con T la collezione di tutti gli insiemi che non sono elementi di se stessi, non si riesce a stabilire se T appartiene o no a se stesso. Per evitare questo genere di paradossi, nel 1903 Russell riconobbe che era necessario formulare la teoria degli insiemi sulla base di un gruppo di assiomi che evitassero definizioni circolari. Come diventerà chiaro con la teoria assiomatica degli insiemi successivamente sviluppata da Zermelo e Fraenkel, non tutte le collezioni di insiemi definibili nel linguaggio usato per la teoria degli insiemi sono insiemi; terminologicamente, si distingue tra classi e insiemi.
- Come conseguenza, agli inizi del 1900 prese piede il logicismo, una scuola di pensiero basata sulla visione di Frege secondo cui la matematica è riducibile alla logica. In particolare, il logicismo di Russell consisteva in due tesi principali. In primo luogo, tutte le verità matematiche possono essere tradotte in verità logiche, cioè il vocabolario della matematica è incluso nel vocabolario della logica. In secondo luogo, tutte le dimostrazioni dei teoremi della matematica possono essere riformulate come prove logiche, cioè l’insieme dei teoremi della matematica è incluso nell’insieme dei teoremi della logica. A supporto del logicismo, tra il 1910 e il 1913 Russell e Whitehead dimostrarono formalmente nel loro libro “Principia Mathematica” il grosso delle conoscenze matematiche del loro tempo utilizzando pure manipolazioni simboliche.
- All’apice del logicismo di Russell, nel 1920 Hilbert lanciò un programma per trovare una singola procedura formale per derivare tutti i teoremi della matematica, che equivale a riuscire a ridurre tutta la matematica in forma assiomatica e a provare che tale assiomatizzazione non porta a contraddizioni. Hilbert sosteneva che “una volta stabilito un formalismo logico, ci si può aspettare che sia possibile un trattamento sistematico, diciamo computazionale, delle formule logiche, in parte analogo alla teoria delle equazioni in algebra”. Secondo Hilbert, le teorie complesse della matematica potevano essere fondate su teorie più semplici fino a basare l’intera matematica sull’aritmetica, cosicché provando la correttezza di quest’ultima si sarebbe provata la non contraddittorietà di tutta la matematica.

- Il colpo di grazia al programma di Hilbert e quindi al logicismo di Russell venne inflitto nel 1931 dai due teoremi di incompletezza dimostrati da Gödel (basati sul metodo della diagonale con cui Cantor mostrò che $|\mathbb{N}| < |\mathbb{R}|$ e su opportune enumerazioni che associano numeri naturali univoci a elementi di insiemi), che esprimiamo informalmente come segue:
 - Primo teorema di incompletezza di Gödel: In ogni teoria assiomatizzabile sufficientemente espressiva da formare affermazioni su ciò che può dimostrare, ci saranno sempre delle affermazioni vere che la teoria può esprimere ma non derivare dai suoi assiomi attraverso le sue regole di inferenza. Ciò significa che in ogni teoria matematica sufficientemente potente ci sono dei teoremi che non sono dimostrabili all'interno della teoria stessa; per dimostrarli, bisognerà ricorrere a una teoria più potente, ma anch'essa soffrirà della stessa limitazione.
 - Secondo teorema di incompletezza di Gödel: Ogni teoria assiomatizzabile sufficientemente espressiva da formare affermazioni sull'aritmetica non potrà mai dimostrare la propria correttezza. Questo risultato evidenzia come già una branca fondamentale della matematica quale l'aritmetica sollevi problemi di incompletezza dal punto di vista di un suo trattamento computazionale.
- Nel 1936 apparvero risultati negativi analoghi in ambito computazionale:
 - Church dimostrò che la logica dei predicati non è decidibile, cioè che non esiste alcun algoritmo in grado di stabilire in tempo finito se una formula predicativa arbitraria è soddisfacibile o meno.
 - Turing dimostrò che il problema della terminazione non è decidibile, cioè che non esiste alcun algoritmo in grado di stabilire in tempo finito se l'esecuzione di un algoritmo arbitrario su un'istanza arbitraria dei suoi dati di ingresso termina oppure no.
- L'esistenza in ogni sistema deduttivo di teoremi che non possono essere dimostrati e l'esistenza di problemi computazionali che non possono essere risolti da nessun algoritmo determinò l'impossibilità di ridurre la matematica alla logica e quindi la fine del logicismo. Dal 1940 in poi la logica continuò a svilupparsi non più come il fondamento universale di tutta la matematica, ma come una branca di essa, pur rimanendo aperta la possibilità di mettere a punto sistemi deduttivi che servano come fondamenti per parti specifiche della matematica.
- La logica è particolarmente importante anche nell'informatica per via della natura computazionale del concetto di deduzione. Facendo un paragone con il ruolo che il calcolo infinitesimale ha avuto nella fisica, a volte la logica viene indicata come il calcolo dell'informatica. A partire dai primi risultati di indecidibilità (anni 1930) e ancor più dall'avvento dei primi elaboratori elettronici (anni 1940) e dei primi linguaggi di programmazione di alto livello (anni 1950), la logica ha avuto un ruolo fondamentale e pervasivo nell'informatica:
 - Nella teoria della calcolabilità e nella teoria della complessità, la logica ha fornito caratterizzazioni del concetto di risolubilità di un problema computazionale sin dall'introduzione del modello astratto di macchina di Turing come pure della visione computazionale della nozione di funzione matematica attraverso il λ -calcolo di Church. Inoltre, la logica è stata cruciale nello sviluppo della teoria della NP-completezza, la quale formalizza il concetto di esplosione combinatoria. Ad esempio, il problema della soddisfacibilità di una formula di logica proposizionale è l'archetipo di problema computazionale che non sappiamo come risolvere in modo efficiente.
 - Nell'architettura degli elaboratori, i circuiti che formano i componenti hardware sono costituiti da porte che, come scoperto da Shannon, implementano le operazioni logiche introdotte da Boole. Sebbene Boole intendesse studiare le leggi del ragionamento della mente umana, l'applicazione principale della sua teoria si trova dunque nell'ingegneria elettronica e nell'industria informatica.
 - Nella progettazione dell'hardware e del software, la logica viene utilizzata per verificare la correttezza di un progetto con un grado di confidenza che va oltre quello del testing. Per la precisione, viene spesso impiegata una variante della logica classica detta logica temporale la quale comprende operatori che consentono di esprimere affermazioni sull'ordine in cui certe computazioni hanno luogo oppure sul fatto che certe computazioni possano o debbano avere luogo.

- Nella gestione delle basi di dati, un linguaggio comunemente impiegato per reperire e manipolare dati è SQL. Questo linguaggio di interrogazione per basi di dati risulta essere un'implementazione della logica del primo ordine.
- Nell'intelligenza artificiale, la logica è essenziale per formalizzare le conoscenze già acquisite e le regole di ragionamento sulla base delle quali i sistemi esperti riescono a dedurre nuove conoscenze. Lo stesso meccanismo si applica ai sistemi per la dimostrazione automatica di teoremi.
- Nella programmazione degli elaboratori, è necessario formalizzare la semantica dei linguaggi di programmazione in modo che i loro costrutti siano coerenti in implementazioni diverse dello stesso linguaggio e vengano compresi senza ambiguità da parte dei programmatori. La logica fornisce strumenti per sviluppare tale semantica e per ragionare sulle proprietà dei programmi (come le triple di Hoare). Inoltre, esistono paradigmi di programmazione – come quello dichiarativo di natura logica di cui il linguaggio Prolog è il principale esponente – in cui la logica viene usata per esprimere i programmi e i suoi meccanismi di ragionamento vengono sfruttati per eseguire i programmi.

8.2 Prolog: Clausole di Horn e Strategia di Risoluzione SLD

- Il linguaggio Prolog (PROgrammation en LOGique) venne sviluppato nel 1972 da Colmerauer e Roussel per l'elaborazione del linguaggio naturale, basandosi sull'interpretazione procedurale delle clausole di Horn data da Kowalski e sul metodo di risoluzione di Robinson. Esso cominciò a essere riconosciuto come un utile strumento di programmazione grazie al compilatore implementato nel 1977 da Warren.
- **Prolog si fonda sulla logica dei predicati limitandosi a formule espresse come clausole di Horn per motivi computazionali.** Un programma Prolog si presenta come una collezione di definizioni di predicati, interpretati come relazioni matematiche, e relative applicazioni e composizioni che danno luogo a fatti e regole. I primi consentono di formalizzare la conoscenza su un certo dominio, mentre le seconde permettono di ricavare ulteriore conoscenza.
- **L'esecuzione di un programma Prolog viene attivata mediante un'interrogazione espressa come vincolo e il suo risultato è un valore di verità.** Se nell'interrogazione sono presenti delle variabili, il risultato è accompagnato dai legami creati via unificazione per tali variabili durante l'applicazione a tempo d'esecuzione dell'algoritmo di refutazione basato sul metodo di risoluzione di Robinson.
- Diversamente dalla programmazione imperativa, in Prolog:
 - I programmi non sono istruzioni di assegnamento da eseguire nell'ordine stabilito dalle istruzioni di controllo del flusso, ma collezioni di predicati da valutare.
 - Le esecuzioni dei programmi non sono sequenze di passi che modificano il contenuto della memoria, ma creano legami immutabili per le variabili e restituiscono valori di verità.
 - Non ci sono effetti collaterali in quanto la valutazione di un predicato non può modificare variabili al di fuori del suo ambiente locale.
 - La conseguente trasparenza referenziale fa sì che venga restituito sempre lo stesso risultato ogni volta che un predicato viene valutato sugli stessi argomenti.
 - La ricorsione è l'unico strumento linguistico per rappresentare l'iterazione e il tipo di dato ricorsivo lista è l'unico tipo di dato strutturato disponibile, con la gestione dinamica della memoria per le liste completamente automatizzata.
- Diversamente da Haskell, in Prolog è possibile definire solo funzioni che restituiscono un valore di verità – seguendo la sintassi delle clausole di Horn con $\rightarrow$ peraltro – e **non esiste un reale sistema di tipi**. D'altro canto, nelle definizioni delle funzioni Prolog (predicati) si esprime solo conoscenza positiva – i casi in cui il risultato è falso non vengono formalizzati in quanto automaticamente derivati dalla mancanza di unificazione – e gli argomenti effettivi possono essere specificati anche solo parzialmente nelle invocazioni – così vengono calcolati i valori di quegli argomenti che producono vero come risultato.
- Il linguaggio Prolog consente di definire delle relazioni e di compiere delle interrogazioni su di esse. Pertanto in Prolog le clausole di Horn costituite da fatti e regole vengono usate per rappresentare informazioni sotto forma di relazioni tra dati. Precisamente, la sintassi di un programma Prolog prevede una sequenza di clausole di Horn date da fatti e regole che sono terminate dal punto:

- Ogni fatto presente nel programma rappresenta un'informazione ed è costituito da una formula atomica della logica dei predicati:
 $P(t_1, \dots, t_n).$
- Ogni regola presente nel programma rappresenta una relazione tra più informazioni:
 $P(t_1, \dots, t_n) :- Q_1(t_{1_1}, \dots, t_{1_{m1}}), \dots, Q_k(t_{k_1}, \dots, t_{k_{mk}}).$
 La parte sinistra della regola è una formula atomica della logica dei predicati detta testa, il simbolo $:-$ rappresenta l'implicazione nel verso $\leftarrow$ e la parte destra della regola è una sequenza non vuota di formule atomiche della logica dei predicati detta corpo. Le virgole che separano le formule presenti all'interno del corpo della regola rappresentano delle congiunzioni.
- Il metodo di risoluzione di Robinson viene impiegato – insieme a un algoritmo di unificazione efficiente – per risolvere problemi definiti attraverso clausole di Horn costituite da vincoli, i quali vengono interpretati come obiettivi da raggiungere istanziandone le parti eventualmente non specificate. Precisamente, l'esecuzione di un programma Prolog viene attivata da un obiettivo descritto come una regola senza testa in cui il simbolo $:-$ è sostituito dal simbolo $?-$, cioè una clausola di Horn data da un vincolo. Il raggiungimento dell'obiettivo viene perseguito applicando il metodo di risoluzione ai fatti e alle regole del programma e all'obiettivo dato. In caso di refutazione, se l'obiettivo non è una clausola ground allora le sue variabili vengono istanziate sulla base delle unificazioni effettuate durante le applicazioni della regola di inferenza per risoluzione.
- In un contesto guidato dagli obiettivi come quello di Prolog, ogni regola di un programma si presta a una duplice interpretazione:
 - Il significato dichiarativo della regola è che la formula di testa $P(t_1, \dots, t_n)$ è vera se le formule del corpo $Q_1(t_{1_1}, \dots, t_{1_{m1}}), \dots, Q_k(t_{k_1}, \dots, t_{k_{mk}})$ sono vere.
 - Il significato procedurale della regola è che per raggiungere l'obiettivo $P(t_1, \dots, t_n)$ bisogna prima raggiungere gli obiettivi $Q_1(t_{1_1}, \dots, t_{1_{m1}}), \dots, Q_k(t_{k_1}, \dots, t_{k_{mk}})$.
- L'algoritmo di refutazione basato sul metodo di risoluzione di Robinson applica ripetutamente la regola di inferenza per risoluzione e questo può generare un numero eccessivo di clausole irrilevanti o ridondanti. Per evitare ciò, si utilizzano delle strategie di risoluzione che scelgono in modo mirato le clausole da cui generare una risolvente. Queste strategie si dividono in complete e incomplete a seconda che riescano sempre a derivare $\square$ da un insieme insoddisfacibile di clausole oppure no. La strategia di risoluzione adottata in Prolog è SLD.
- Dato un insieme di clausole C , diciamo che una dimostrazione per risoluzione $c_1 \dots c_n$ a partire da C è ottenuta applicando:
 - La strategia di risoluzione lineare sse per ogni $i = 2, \dots, n$ la clausola c_i è la risolvente della clausola precedente c_{i-1} e di una clausola $c' \in C \cup \{c_1, \dots, c_{i-1}\}$.
 - La strategia di risoluzione di input sse per ogni $i = 2, \dots, n$ la clausola c_i è la risolvente di una clausola di ingresso $c \in C$ e di una clausola $c' \in C \cup \{c_1, \dots, c_{i-1}\}$.
 - La strategia di risoluzione SLD sse per ogni $i = 2, \dots, n$ la clausola c_i è la risolvente della clausola precedente c_{i-1} e di una clausola di ingresso $c \in C$ (combinazione efficiente di risoluzione lineare e risoluzione di input).
- Teorema: La strategia di risoluzione lineare è completa per tutti gli insiemi di clausole.
- Teorema: La strategie di risoluzione di input ed SLD sono complete per gli insiemi di clausole di Horn.
- Dati un programma Prolog (insieme di clausole C) e un obiettivo (clausola iniziale c_1), in virtù dell'interpretazione procedurale e della strategia di risoluzione SLD bisogna trovare all'interno del programma, per tutte le formule atomiche nell'obiettivo considerate da sinistra a destra, dei fatti che siano unificabili con quelle formule o delle regole le cui teste siano unificabili con quelle formule. Nel caso la risoluzione avvenga con una regola, le formule atomiche nel corpo della regola entrano a far parte del nuovo obiettivo. Per motivi di efficienza, conviene che le regole ricorsive siano ricorsive a destra, cioè che i predicati presenti nella testa di tali regole compaiano nelle formule atomiche più a destra nel corpo delle regole stesse.

- Poiché ogni formula atomica presente in un obiettivo potrebbe essere unificabile con più fatti e teste di regole di un programma, in Prolog la risoluzione della formula atomica più a sinistra in un obiettivo viene esaminata rispetto a tutte le clausole del programma considerate nell'ordine in cui sono scritte. Per motivi di efficienza, all'interno dei programmi conviene quindi elencare i fatti prima delle regole e le regole non ricorsive prima di quelle ricorsive.
- L'esecuzione di un programma Prolog su un obiettivo può essere rappresentata graficamente tramite un albero e-o, così chiamato perché ogni nodo rappresenta un obiettivo (coniunzione) e ha tanti nodi figli quanti sono i fatti e le regole alternativi tra loro con i quali può avvenire la risoluzione (disgiunzione). Durante l'applicazione della risoluzione, l'albero viene costruito in profondità per via dell'ordine LIFO imposto dalla strategia SLD.
- La radice dell'albero e-o contiene l'obiettivo iniziale, mentre ogni foglia può contenere $\square$ oppure un obiettivo alla cui formula atomica più a sinistra non è applicabile la risoluzione. Ogni ramo dell'albero è etichettato con i legami creati via unificazione durante la corrispondente applicazione della regola di inferenza per risoluzione. Ogni cammino dalla radice a una foglia contenente $\square$ è un cammino di successo, mentre tutti gli altri cammini sono cammini di fallimento o cammini infiniti (considerare le clausole di un programma Prolog sempre nello stesso ordine rende la strategia SLD incompleta).

- Esempio: Consideriamo il seguente programma Prolog:

```
genitore(giovanni, andrea).
genitore(andrea, paolo).
antenato(X, Y) :- genitore(X, Y).
antenato(X, Y) :- genitore(X, Z), antenato(Z, Y).
```

dove il predicato `genitore(X, Y)` (risp. `antenato(X, Y)`) indica che `X` è genitore (risp. antenato) di `Y`. Osservato che ogni fatto o regola contenente delle variabili ha una quantificazione universale implicita su tutte quelle variabili, le due regole del programma dato vanno intese nel seguente modo:

$$\forall X \forall Y (genitore(X, Y) \rightarrow antenato(X, Y))$$

$$\forall X \forall Y \forall Z (genitore(X, Z) \wedge antenato(Z, Y) \rightarrow antenato(X, Y))$$

Consideriamo poi l'obiettivo:

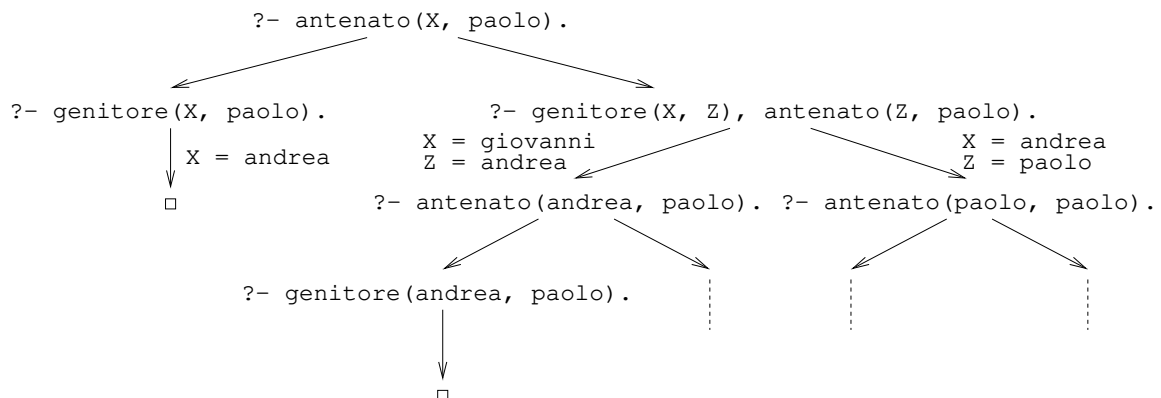
```
?- antenato(X, paolo).
```

Osservato che ogni obiettivo contenente delle variabili ha una quantificazione esistenziale implicita su tutte quelle variabili, l'obiettivo dato va inteso nel seguente modo:

$$\exists X antenato(X, paolo) ?$$

che dal punto di vista procedurale consiste nel trovare un antenato della persona specificata sulla base delle informazioni fornite dal programma. Poiché l'esecuzione di un programma Prolog procede per refutazione dell'obiettivo, l'obiettivo deve essere negato prima di iniziare ad applicare la regola di inferenza per risoluzione e quindi la quantificazione esistenziale che precede eventualmente l'obiettivo diventa universale.

L'albero e-o risultante è il seguente:



in cui si nota la presenza di due cammini di successo (soluzioni `X = andrea` e `X = giovanni`) assieme a diversi cammini che conducono al fallimento. ■ftplf_18

8.3 Prolog: Sintassi dei Termini e Predicati Predefiniti

- I caratteri utilizzabili all'interno di un programma Prolog comprendono le 26 lettere minuscole, le 26 lettere maiuscole, le 10 cifre decimali, il sottotratto, la barra verticale, i simboli di punteggiatura, le parentesi tonde e quadre, gli operatori aritmetici e relazionali e i caratteri di spaziatura. Le lettere minuscole sono considerate diverse dalle corrispondenti lettere maiuscole (case sensitivity).
- I caratteri possono essere combinati per formare argomenti di predicati così classificati:
 - Atomi che si dividono in:
 - * Costanti costituite da:
 - numeri interi e reali;
 - sequenze di lettere, cifre e sottotratti che iniziano con una minuscola;
 - sequenze racchiuse tra apici di lettere, cifre e sottotratti che iniziano con una maiuscola.
 - * Variabili costituite da:
 - sequenze di lettere, cifre e sottotratti che iniziano con una maiuscola;
 - singoli sottotratti `_`, che rappresentano un qualsiasi valore.
 - Termini composti costituiti dalla concatenazione di:
 - * Funtori che sono espressi come le costanti non numeriche, cioè costituiti da:
 - sequenze di lettere, cifre e sottotratti che iniziano con una minuscola;
 - sequenze racchiuse tra apici di lettere, cifre e sottotratti che iniziano con una maiuscola.
 - * Argomenti separati da virgole e racchiusi tra parentesi tonde, costituiti da:
 - atomi;
 - termini composti.
 - Dati strutturati che si dividono in:
 - * Liste costituite da sequenze di elementi separati da virgole e racchiusi tra parentesi quadre, dotate di un operatore `|` per distinguere un certo numero di elementi iniziali dai successivi.
 - * Stringhe costituite da sequenze di caratteri racchiusi tra virgolette, implementate come liste.
- Esempi: 10 e -13.5 sono costanti numeriche, `gatto` e `'Proprietario_di'` sono costanti simboliche o funtori, `X` e `Risultato_operazione` sono variabili, `"ciao!"` e `"Divisione illegale"` sono stringhe, `[]` e `[1, 2, 3]` sono liste.
- I predicati, i cui nomi seguono le stesse convenzioni dei funtori, sono definiti, spesso ricorsivamente, tramite sequenze di fatti e regole. Eventuali commenti devono essere racchiusi tra `/*` e `*/` oppure iniziare con `%` ed estendersi su una singola linea.
- Esempi relativi alle liste:

- Predicato per stabilire se una sequenza di caratteri è una lista:

```
lista([]).
lista([X | L]).                                /* lista([_ | _]). */
```

I due fatti specificano le sequenze di caratteri che sono conformi alla sintassi delle liste in Prolog. Qualsiasi altra sequenza non unificabile col formato degli argomenti nei due fatti non è una lista.

- Predicato per stabilire se un elemento appartiene a una lista (la lista vuota non va bene come caso base della ricorsione perché rappresenta la situazione in cui l'elemento non appartiene alla lista, mentre in Prolog si esprime solo conoscenza positiva):

```
membro(X, [X | L]).                            /* membro(X, [X | _]). */
membro(X, [Y | L]) :- membro(X, L).           /* membro(X, [_ | L]) :- membro(X, L). */
```

Dati gli obiettivi:

```
?- membro(2, []).
?- membro(2, [0, 2, 4]).
?- membro(Z, [1, 3, 5]).
```

abbiamo che il primo fallisce, il secondo ha successo e il terzo dà luogo a tanti cammini di successo (ciascuno con la propria istanziazione di `Z`) quanti sono gli elementi presenti nella lista. Dunque, a seconda di come viene specificato l'obiettivo, questo predicato può essere usato sia per cercare un elemento in una lista, sia per attraversare tutti gli elementi di una lista.

- Predicato per stabilire se una lista è un prefisso di un'altra lista (cioè compare all'inizio; nella prima clausola bisogna controllare se L è una lista perché Prolog non ha un sistema di tipi):


```

      prefisso([], L)                :- lista(L).
      prefisso([X | L1], [X | L2]) :- prefisso(L1, L2).
      
```
- Predicato per stabilire se una lista è un suffisso di un'altra lista (cioè compare alla fine):


```

      suffisso([], L)                :- lista(L).
      suffisso(L, L)                :- lista(L).
      suffisso(L1, [_ | L2]) :- suffisso(L1, L2).
      
```
- Predicato per stabilire se una lista è una sottolista di un'altra lista (cioè compare all'interno):


```

      sottolista(L1, L2)            :- prefisso(L1, L2).
      sottolista(L1, [_ | L2]) :- sottolista(L1, L2).
      
```
- Predicato per aggiungere un elemento all'inizio di una lista:


```

      inserisci_elem(X, L, [X | L]).
      
```
- Predicato per aggiungere un elemento alla fine di una lista:


```

      accoda_elem(X, [], [X]).
      accoda_elem(X, [Y | L], [Y | LX]) :- accoda_elem(X, L, LX).
      
```
- Predicato per concatenare due liste:


```

      concatena(L, [], L)           :- lista(L).
      concatena(L1, [X | L2], L) :- accoda_elem(X, L1, L1X), concatena(L1X, L2, L).
      
```
- Predicato per invertire l'ordine degli elementi di una lista:


```

      inverti([], []).
      inverti([X | L1], L2) :- inverti(L1, L1Inv), accoda_elem(X, L1Inv, L2).
      
```
- Al fine di agevolare l'attività di programmazione, Prolog mette a disposizione simboli e predicati predefiniti. Tra questi abbiamo i consueti operatori aritmetici e relazionali in notazione infissa, su termini di tipo numerico, che abilitano la valutazione di espressioni aritmetiche in un contesto logico:
 - I simboli +, -, *, /, mod, ^ rappresentano rispettivamente le operazioni di addizione, sottrazione, moltiplicazione, divisione, resto della divisione, elevamento a potenza su operandi numerici (sono funtori, non predicati; tutte le variabili eventualmente presenti nei due operandi devono essere già state istanziate con valori numerici).
 - I simboli ==, \=, >, >=, <, <= rappresentano rispettivamente i predicati di uguale-a, diverso-da, maggiore-di, maggiore-uguale, minore-di, minore-uguale su operandi numerici (tutte le variabili eventualmente presenti nei due operandi devono essere già state istanziate con valori numerici).
 - Il predicato is ha successo sse il suo operando sinistro è unificabile con il valore numerico del suo operando destro (tutte le variabili eventualmente presenti nell'operando destro devono essere già state istanziate con valori numerici). Questo è l'unico predicato in Prolog che consente di valutare un'espressione aritmetica e di legarne il valore a una variabile.
- I seguenti predicati in notazione infissa costituiscono delle estensioni dei predicati di uguaglianza a termini di tipo arbitrario (quindi non necessariamente numerico):
 - Il predicato = ha successo sse i suoi due operandi sono unificabili e in tal caso produce il loro upg (non effettua però l'occur check).
 - Il predicato \= ha successo sse i suoi due operandi non sono unificabili.
 - Il predicato == ha successo sse i suoi due operandi sono sintatticamente identici (delle variabili istanziate eventualmente presenti nei due operandi viene considerato il valore anziché il nome).
 - Il predicato \== ha successo sse i suoi due operandi non sono sintatticamente identici.

- Esempi:

?- X is 1 + 2.	?- 4 - 1 == 1 + 2.	?- a < -30.	?- X = 1 + 2.
yes X=3	yes	domain error	yes X=1+2
?- X is Y + 2.	?- 2.5 > 9.	?- 2 = 2.	?- f(a, Y) = f(Z, g(7)).
instantiation error	no	yes	yes Z=a, Y=g(7)
?- 3 is 1 + 2.	?- Z < 15.	?- a = b.	?- X == 2.
yes	instantiation error	no	no
?- 4 - 1 is 1 + 2.	?- Z is 4, Z < 15.	?- a = A.	?- X = 2, X == 2.
no	yes Z=4	yes A=a	yes X=2

- Esempi relativi alle funzioni numeriche (se il risultato di una funzione con n argomenti non è un valore di verità, allora il corrispondente predicato in Prolog ha $n+1$ argomenti dei quali l'ultimo è il risultato; inoltre ogni argomento effettivo il cui valore deriva da un'espressione va prima preparato con `is`):

- Predicato per calcolare il fattoriale di un numero naturale:

```
fattoriale(0, 1).
fattoriale(N, F) :- N > 0, N1 is N - 1,
                    fattoriale(N1, F1), F is N * F1.
```

- Predicato per calcolare l' n -esimo numero di Fibonacci:

```
fibonacci(0, 0).
fibonacci(1, 1).
fibonacci(N, F) :- N > 1, N1 is N - 1, N2 is N - 2,
                    fibonacci(N1, F1), fibonacci(N2, F2), F is F1 + F2.
```

- Ulteriori esempi relativi alle liste:

- Predicato per calcolare il numero di elementi presenti in una lista:

```
lunghezza([], 0).
lunghezza(_ | L, N) :- lunghezza(L, M), N is M + 1.
```

- Il predicato che stabilisce se un elemento appartiene a una lista può essere reso più efficiente ridefinendolo come segue:

```
membro(X, [X | _]).
membro(X, [_ | L]) :- X \== Y, membro(X, L).
```

In questo modo le due clausole sono alternative tra loro e quindi al più una di esse può dare luogo a un cammino di successo. Per esempio, nel caso in cui l'obiettivo sia il seguente:

```
?- membro(1, [3, 1, 5, 1, 7]).
```

c'è un unico cammino di successo che termina quando il secondo argomento diventa `[1, 5, 1, 7]`. La vecchia definizione del predicato avrebbe invece avuto due cammini di successo.

- Predicato per rimuovere tutte le occorrenze di un elemento da una lista:

```
rimuovi_elem(_, [], []).
rimuovi_elem(X, [X | L], LNoX) :- rimuovi_elem(X, L, LNoX).
rimuovi_elem(X, [_ | L], [Y | LNoX]) :- X \== Y, rimuovi_elem(X, L, LNoX).
```

- Predicato per fondere ordinatamente due liste ordinate di valori numerici:

```
fondi(L, [], L) :- lista(L).
fondi([], L, L) :- L \== [], lista(L).
fondi([X | L1], [Y | L2], [X | L]) :- X < Y, fondi(L1, [Y | L2], L).
fondi([X | L1], [Y | L2], [X, Y | L]) :- X == Y, fondi(L1, L2, L).
fondi([X | L1], [Y | L2], [Y | L]) :- X > Y, fondi([X | L1], L2, L).
```

- Esempi relativi agli algoritmi di ordinamento:

– Mergesort:

```
mergesort([], []).
mergesort([X], [X]).
mergesort([X1, X2 | L], L0rd) :- dimezza([X1, X2 | L], L1, L2),
                                mergesort(L1, L10rd), mergesort(L2, L20rd),
                                fondi(L10rd, L20rd, L0rd).

dimezza([], [], []).
dimezza([X], [X], []).
dimezza([X1, X2 | L], [X1 | L1], [X2 | L2]) :- dimezza(L, L1, L2).
```

– Quicksort:

```
quicksort([], []).
quicksort([P | L], L0rd) :- partiziona(L, P, LInf, LSup),
                            quicksort(LInf, LInf0rd), quicksort(LSup, LSup0rd),
                            concatena(LInf0rd, [P | LSup0rd], L0rd).

partiziona([], _, [], []).
partiziona([X | L], P, [X | LInf], LSup) :- X < P, partiziona(L, P, LInf, LSup).
partiziona([X | L], P, LInf, [X | LSup]) :- X >= P, partiziona(L, P, LInf, LSup).
```

- Prolog mette a disposizione dei predicati predefiniti per discriminare tra differenti tipi di termini:

- `var(T)` ha successo sse `T` è una variabile non istanziata, cioè una variabile non legata a un valore.
- `nonvar(T)` ha successo sse `T` non è una variabile non istanziata.
- `integer(T)` ha successo sse `T` è istanziato con una costante numerica intera.
- `float(T)` ha successo sse `T` è istanziato con una costante numerica reale.
- `number(T)` ha successo sse `T` è istanziato con una costante numerica.
- `atom(T)` ha successo sse `T` è istanziato con una costante non numerica.
- `compound(T)` ha successo sse `T` è istanziato con un termine composto:
 - * `functor(T, F, N)` ha successo sse `T` è un termine composto il cui funtore ha nome `F` ed è applicato a `N` argomenti.
 - * `arg(N, T, A)` ha successo sse `T` è un termine composto il cui `N`-esimo argomento è `A`.

- Ulteriori esempi relativi alle funzioni numeriche:

– Predicato per calcolare quoziente e resto della divisione tra due numeri naturali:

```
divisione(X, Y, Q, R) :- integer(X), X >= 0, integer(Y), Y > 0,
                        Q is X / Y, R is X mod Y.
```

– Predicato per calcolare la somma o la differenza a seconda di quali operandi risultano istanziati (l'uso del predicato `is` li costringe a essere istanziati con valori numerici):

```
somma_diff(X, Y, Z) :- nonvar(X), nonvar(Y), Z is X + Y.
somma_diff(X, Y, Z) :- nonvar(X), nonvar(Z), Y is Z - X.
somma_diff(X, Y, Z) :- nonvar(Y), nonvar(Z), X is Z - Y.
```

– Predicato per calcolare il prodotto o il quoziente a seconda di quali operandi risultano istanziati (il valore numerico di `Y` nella terza regola e di `X` nella quinta regola è irrilevante ma deve essere presente, da cui il controllo finale `number` su quella variabile):

```
prod_quoz(X, Y, Z) :- nonvar(X), nonvar(Y), Z is X * Y.
prod_quoz(X, Y, Z) :- nonvar(X), nonvar(Z), X =\= 0, Y is Z / X.
prod_quoz(X, Y, Z) :- nonvar(X), nonvar(Z), X =:= 0, Z =:= 0, number(Y).
prod_quoz(X, Y, Z) :- nonvar(Y), nonvar(Z), Y =\= 0, X is Z / Y.
prod_quoz(X, Y, Z) :- nonvar(Y), nonvar(Z), Y =:= 0, Z =:= 0, number(X).
```

- Esempi relativi a termini e loro unificazione:

- Predicato per stabilire se un termine è ground, cioè privo di variabili non istanziate (quando è composto, bisogna verificare che tutti i suoi argomenti siano ground):

```
ground(T) :- integer(T).
ground(T) :- float(T).
ground(T) :- atom(T).
ground(T) :- compound(T), functor(T, _, N), ground_tutti_arg(N, T).
ground_tutti_arg(0, _).
ground_tutti_arg(N, T) :- N > 0, arg(N, T, A), ground(A),
                           N1 is N - 1, ground_tutti_arg(N1, T).
```

- Predicato per stabilire se un termine è sottoterminale di un altro (`sottoterm(T, T)` non va bene come caso base quando uno dei due termini è una variabile perché verrebbe unificato con l'altro; quando il secondo è composto, bisogna verificare che il primo sia sottoterminale di almeno uno degli argomenti del secondo):

```
sottoterm(T1, T2) :- T1 == T2.
sottoterm(T1, T2) :- T1 \== T2, compound(T2),
                      functor(T2, _, N), sottoarg(T1, N, T2).
sottoarg(T1, N, T2) :- arg(N, T2, A), sottoterm(T1, A).
sottoarg(T1, N, T2) :- N > 1, N1 is N - 1, sottoarg(T1, N1, T2).
```

- Algoritmo di unificazione di Robinson (stesso effetto del predicato = ma con occur check):

```
unifica(T1, T2, []) :- integer(T1), integer(T2), T1 == T2.
unifica(T1, T2, []) :- float(T1), float(T2), T1 == T2.
unifica(T1, T2, []) :- atom(T1), atom(T2), T1 == T2.
unifica(T1, T2, [[T1, T2]]) :- var(T1), var(T2).
unifica(T1, T2, [[T2, T1]]) :- var(T1), integer(T2).
unifica(T1, T2, [[T2, T1]]) :- var(T1), float(T2).
unifica(T1, T2, [[T2, T1]]) :- var(T1), atom(T2).
unifica(T1, T2, [[T2, T1]]) :- var(T1), compound(T2), occur_check(T1, T2).
unifica(T1, T2, [[T1, T2]]) :- var(T2), integer(T1).
unifica(T1, T2, [[T1, T2]]) :- var(T2), float(T1).
unifica(T1, T2, [[T1, T2]]) :- var(T2), atom(T1).
unifica(T1, T2, [[T1, T2]]) :- var(T2), compound(T1), occur_check(T2, T1).
unifica(T1, T2, L) :- compound(T1), compound(T2),
                      functor(T1, F, N), functor(T2, F, N),
                      unifica_tutti_arg(N, T1, T2, L).

unifica_tutti_arg(0, _, _, []).
unifica_tutti_arg(N, T1, T2, L) :- N > 0, arg(N, T1, A1), arg(N, T2, A2),
                                   unifica(A1, A2, LA),
                                   N1 is N - 1, unifica_tutti_arg(N1, T1, T2, L1),
                                   coerenti(L1, LA), concatena(L1, LA, L).

coerenti(_, []).
coerenti(L1, [[T, X] | L2]) :- no_stessa_v_diff_t(X, T, L1), coerenti(L1, L2).
no_stessa_v_diff_t(_, _, []).
no_stessa_v_diff_t(X, T, [[_, Y] | L]) :- X \== Y, no_stessa_v_diff_t(X, T, L).
no_stessa_v_diff_t(X, T, [[T, X] | L]) :- no_stessa_v_diff_t(X, T, L).

occur_check(_, T) :- integer(T).
occur_check(_, T) :- float(T).
occur_check(_, T) :- atom(T).
occur_check(X, T) :- var(T), X \== T.
occur_check(X, T) :- compound(T), functor(T, _, N), occur_check_tutti_arg(X, N, T).
occur_check_tutti_arg(_, 0, _).
occur_check_tutti_arg(X, N, T) :- N > 0, arg(N, T, A), occur_check(X, A),
                                   N1 is N - 1, occur_check_tutti_arg(X, N1, T).
```

8.4 Prolog: Negazione, Taglio, Input/Output, Predicati Avanzati

- Le formule atomiche presenti nel corpo delle regole di un programma Prolog e negli obiettivi sono tutte positive perché questo è il formato delle clausole di Horn con $\rightarrow$. A volte sarebbe tuttavia utile poter esprimere anche formule atomiche negative. A tale scopo Prolog mette a disposizione il predicato `not` la cui interpretazione è conforme all'assunzione di mondo chiuso: è vero solo ciò che è stabilito dai fatti del programma o che può essere dedotto applicando le regole del programma, tutto il resto è falso.
- `not(T)`, a volte indicato con `\+(T)`, ha successo sse `T` fallisce, cioè Prolog implementa la negazione come fallimento. A causa della possibile presenza di cammini infiniti nell'albero e-o di `T`, la regola di negazione come fallimento è un'approssimazione dell'assunzione di mondo chiuso e la negazione così implementata non coincide con la negazione della logica matematica. La regola di negazione come fallimento è corretta e completa quando l'argomento `T` è ground, cioè privo di variabili non istanziate.
- Esempi:

- Dati il seguente programma Prolog:

```
studente(franco).
sposato(roberto).
studente_celibe(X) :- studente(X), not(sposato(X)).
```

e il seguente obiettivo col quale si chiede se esiste uno studente celibe:

```
?- studente_celibe(X).
```

abbiamo successo con `X` istanziata a `franco`. Osserviamo che quando viene considerata la formula atomica `not(sposato(X))` durante l'esecuzione, la variabile `X` è già stata istanziata e quindi l'argomento del `not` è assimilabile a un termine ground.

Se invece la regola del programma fosse stata espressa nel seguente modo:

```
studente_celibe(X) :- not(sposato(X)), studente(X).
```

allora l'argomento del `not` non sarebbe stato assimilabile a un termine ground. Poiché `X` sarebbe stata istanziata a `roberto`, avremmo erroneamente avuto un fallimento. Ciò mostra che la regola della negazione come fallimento non è sempre corretta e completa quando l'argomento del `not` non è ground.

- L'occur check può essere riformulato tramite la singola clausola:

```
occur_check(X, T) :- not(sottoterm(X, T)).
```

a patto di invocarla con `T` ground.

- Dati un programma Prolog e un obiettivo, la costruzione in profondità del corrispondente albero e-o viene sospesa quando si raggiunge la foglia di un cammino di successo. A quel punto l'interprete chiede all'utente se vuole terminare l'esecuzione oppure se vuole continuare l'esecuzione in cerca di altre soluzioni del problema. In quest'ultimo caso, l'esecuzione riprende risalendo dalla foglia verso la radice fino a incontrare il primo nodo la cui formula atomica più a sinistra ammette un'unificazione alternativa a quelle che avevano condotto ai cammini precedentemente intrapresi.
- Poiché non è noto a priori se l'esecuzione di un programma Prolog rispetto a un obiettivo avrà successo, la costruzione in profondità del corrispondente albero e-o equivale ad agire per tentativi e revoche (backtrack). Prima si tenta col cammino più a sinistra. Se questo è un cammino di fallimento oppure è un cammino di successo ma l'utente vuole trovare ulteriori soluzioni al problema, allora si torna indietro sui passi appena compiuti annullando gli eventuali legami creati per unificazione e poi si tenta col cammino successivo e così via per i rimanenti cammini. L'esecuzione fallisce sse tutti i cammini sono cammini di fallimento.

- Prolog mette a disposizione i seguenti predicati predefiniti di natura logica che hanno un impatto sulla costruzione dell'albero e-o:

- **true** ha sempre successo e quindi fa avanzare la costruzione del cammino.
- **fail** non ha mai successo e quindi blocca la costruzione del cammino.
- **!**, detto taglio, ha successo una e una sola volta e ha l'effetto collaterale di bloccare alcuni cammini. Il taglio ha lo scopo di ottenere prestazioni migliori e di evitare soluzioni ridondanti, ma va usato con attenzione perché la sua introduzione nelle clausole di un programma potrebbe alterare l'insieme delle soluzioni di un problema rappresentato da un certo obiettivo.

- Supponiamo che un nodo di un albero e-o contenga l'obiettivo $Q_1, \dots, Q_h, !, Q_{h+1}, \dots, Q_k$, che il taglio evidenziato nell'obiettivo sia stato introdotto per via dell'ultima risoluzione effettuata e che da questo nodo parta un cammino che porta al nodo contenente l'obiettivo $!, Q_{h+1}, \dots, Q_k$. Giunti a quest'ultimo nodo, il taglio ha successo e vengono bloccati tutti gli eventuali cammini che iniziano dai fratelli destri dei nodi che si trovano lungo il cammino tra i due nodi che contengono i due obiettivi considerati. Poi si procede con l'obiettivo rimanente $Q_{h+1}, \dots, Q_k$. Quando l'esecuzione torna indietro al nodo contenente l'obiettivo $!, Q_{h+1}, \dots, Q_k$ a causa di fallimento o della ricerca di ulteriori soluzioni al problema, l'esecuzione passa direttamente al nonno del nodo contenente l'obiettivo $Q_1, \dots, Q_h, !, Q_{h+1}, \dots, Q_k$ in virtù del fatto che il taglio ha già avuto successo e che tutti gli eventuali cammini che iniziano dai fratelli destri dei nodi che si trovano lungo il cammino tra i due nodi che contengono i due obiettivi considerati sono stati bloccati (v. Fig. 8.1).

- Esempi:

- Il predicato che stabilisce se un elemento appartiene a una lista può essere reso più efficiente (nel senso di evitare di unificare anche con `membro(X, [Y | L])` e subito dopo scoprire che `X \== Y` fallisce) ridefinendolo come segue:

```
membro(X, [X | _]) :- !.
membro(X, [_ | L]) :- membro(X, L).
```

Tuttavia osserviamo che l'obiettivo:

```
?- membro(Z, [1, 3, 5]).
```

dà luogo a un solo cammino di successo (in cui **Z** viene istanziata a 1) a causa del taglio, quindi il predicato non può più essere usato per attraversare tutti gli elementi di una lista.

- Predicato per implementare l'istruzione if-then-else (l'ordine tra le due regole non è importante):

```
if_then_else(C, I1, _) :- C, !, I1.
if_then_else(C, _, I2) :- not(C), !, I2.
```

- Il predicato predefinito **not**, che ha successo se il suo argomento fallisce, è definito come segue (sono rilevanti sia l'ordine tra **!** e **fail** nella prima regola che l'ordine tra le due regole):

```
not(T) :- T, !, fail.
not(T).
```

Essa equivale a:

```
not(T) :- T, !, fail; true.
```

dove il punto e virgola rappresenta una disgiunzione, ha meno priorità della virgola ed è comodo per accorpare in una sola più clausole aventi la stessa testa.

- Esempi relativi agli alberi binari (rappresentati attraverso un funtore `ab` dove il primo argomento è un numero intero associato alla radice, il secondo argomento è il sottoalbero sinistro e il terzo argomento è il sottoalbero destro; l'atomo `nil` rappresenta l'albero vuoto):
 - Predicato per stabilire se una sequenza di caratteri è un albero binario:


```
albero_bin(nil).
albero_bin(ab(N, Sx, Dx)) :- integer(N), albero_bin(Sx), albero_bin(Dx).
```
 - Predicato per stabilire se un elemento appartiene a un albero binario visitando quest'ultimo in ordine anticipato:


```
cerca_albero_bin_ant(N, ab(N, _, _)) :- !.
cerca_albero_bin_ant(N, ab(_, Sx, _)) :- cerca_albero_bin_ant(N, Sx).
cerca_albero_bin_ant(N, ab(_, _, Dx)) :- cerca_albero_bin_ant(N, Dx).
```
 - Predicato per stabilire se un elemento appartiene a un albero binario visitando quest'ultimo in ordine posticipato:


```
cerca_albero_bin_post(N, ab(_, Sx, _)) :- cerca_albero_bin_post(N, Sx).
cerca_albero_bin_post(N, ab(_, _, Dx)) :- cerca_albero_bin_post(N, Dx).
cerca_albero_bin_post(N, ab(N, _, _)) :- !.
```
 - Predicato per stabilire se un elemento appartiene a un albero binario visitando quest'ultimo in ordine simmetrico:


```
cerca_albero_bin_simm(N, ab(_, Sx, _)) :- cerca_albero_bin_simm(N, Sx).
cerca_albero_bin_simm(N, ab(N, _, _)) :- !.
cerca_albero_bin_simm(N, ab(_, _, Dx)) :- cerca_albero_bin_simm(N, Dx).
```
 - Predicato per stabilire se una sequenza di caratteri è un albero binario di ricerca:


```
albero_bin_ric(nil).
albero_bin_ric(ab(N, Sx, Dx)) :- albero_bin(ab(N, Sx, Dx)),
                                minore_ug(Sx, N), maggiore_ug(Dx, N).

minore_ug(nil, _).
minore_ug(ab(M, Sx, Dx), N) :- M <= N, minore_ug(Sx, N), minore_ug(Dx, N).
maggiore_ug(nil, _).
maggiore_ug(ab(M, Sx, Dx), N) :- M >= N, maggiore_ug(Sx, N), maggiore_ug(Dx, N).
```
 - Predicato per stabilire se un elemento appartiene a un albero binario di ricerca:


```
cerca_albero_bin_ric(N, ab(N, _, _)) :- !.
cerca_albero_bin_ric(N, ab(M, Sx, _)) :- N < M, cerca_albero_bin_ric(N, Sx).
cerca_albero_bin_ric(N, ab(M, _, Dx)) :- N > M, cerca_albero_bin_ric(N, Dx).
```
- Prolog mette a disposizione dei predicati predefiniti per effettuare operazioni di input/output:
 - `read(T)` ha successo sse è possibile acquisire tramite tastiera una sequenza di caratteri che finisce con il punto (l'acquisizione termina premendo il tasto invio) e questa sequenza escluso il punto è unificabile con `T`.
 - `write(T)` ha successo stampando `T` sullo schermo.
 - `nl` ha successo stampando un'andata a capo sullo schermo.
- Esempi:
 - Predicato per stampare un valore e andare a capo sullo schermo:


```
write_nl(T) :- write(T), nl.
```
 - Predicato per risolvere il problema delle torri di Hanoi (dischi, partenza, arrivo, intermedia):


```
hanoi(1, P, A, _) :- stampa_mossa(P, A).
hanoi(N, P, A, I) :- N > 1, N1 is N - 1,
                    hanoi(N1, P, I, A), stampa_mossa(P, A), hanoi(N1, I, A, P).
stampa_mossa(P, A) :- write('sposta da '), write(P), write(' a '), write_nl(A).
```

- Prolog mette a disposizione dei predicati predefiniti di secondo ordine per raccogliere tutte le soluzioni di un problema rappresentato da un obiettivo (anziché calcolarle una alla volta):
 - `findall(T, O, L)` ha successo sse `L` è unificabile con la lista con eventuali ripetizioni di tutte le istanze del termine `T` per le quali l'obiettivo `O` ha successo.
 - `bagof(T, O, L)` ha successo sse `L` è unificabile con la lista non vuota con eventuali ripetizioni di tutte le istanze del termine `T` per le quali l'obiettivo `O` ha successo, dove le istanze sono organizzate in base alle variabili eventualmente presenti in `O`.
 - `setof(T, O, L)` ha successo sse `L` è unificabile con la lista non vuota senza ripetizioni di tutte le istanze del termine `T` per le quali l'obiettivo `O` ha successo, dove le istanze sono organizzate in base alle variabili eventualmente presenti in `O`.
- Esempi relativi ai grafi diretti (rappresentati attraverso un funtore `gd` dove il primo argomento è una lista senza ripetizioni di atomi che denotano i vertici mentre il secondo argomento è una lista senza ripetizioni di termini basati sul funtore `arco`, i cui due argomenti sono rispettivamente il vertice da cui l'arco esce e il vertice in cui l'arco entra):

- Predicato per stabilire se una sequenza di caratteri è un grafo diretto:

```

grafo_dir(gd(LV, LA)) :- controlla_v(LV), controlla_a(LA, LV).
controlla_v([]).
controlla_v([V | LV]) :- not(membro(V, LV)), controlla_v(LV).
controlla_a([], _).
controlla_a([arco(V1, V2) | LA], LV) :- membro(V1, LV), membro(V2, LV),
                                         not(membro(arco(V1, V2), LA)),
                                         controlla_a(LA, LV).

```

- Predicato per stabilire se un elemento appartiene a un grafo diretto visitando quest'ultimo in ampiezza a partire da un certo vertice iniziale (il secondo e il quarto argomento di `cerca_amp` rappresentano rispettivamente la lista dei vertici da visitare e la lista dei vertici già visitati):

```

cerca_grafo_dir_amp(V, gd(LV, LA), I) :- membro(I, LV), cerca_amp(V, [I], LA, []).
cerca_amp(V, [V | _], _, _) :- !.
cerca_amp(V, [V1 | LV], LA, LVVis) :- V \== V1, membro(V1, LVVis),
                                         cerca_amp(V, LV, LA, LVVis).
cerca_amp(V, [V1 | LV], LA, LVVis) :- V \== V1, not(membro(V1, LVVis)),
                                         findall(V2, membro(arco(V1, V2), LA), LVAdiac),
                                         concatena(LV, LVAdiac, NuovaLV),
                                         cerca_amp(V, NuovaLV, LA, [V1 | LVVis]).

```

- Predicato per stabilire se un elemento appartiene a un grafo diretto visitando quest'ultimo in profondità a partire da un certo vertice iniziale (il secondo e il quarto argomento di `cerca_prof` rappresentano rispettivamente la lista dei vertici da visitare e la lista dei vertici già visitati):

```

cerca_grafo_dir_prof(V, gd(LV, LA), I) :- membro(I, LV), cerca_prof(V, [I], LA, []).
cerca_prof(V, [V | _], _, _) :- !.
cerca_prof(V, [V1 | LV], LA, LVVis) :- V \== V1, membro(V1, LVVis),
                                         cerca_prof(V, LV, LA, LVVis).
cerca_prof(V, [V1 | LV], LA, LVVis) :- V \== V1, not(membro(V1, LVVis)),
                                         findall(V2, membro(arco(V1, V2), LA), LVAdiac),
                                         concatena(LVAdiac, LV, NuovaLV),
                                         cerca_prof(V, NuovaLV, LA, [V1 | LVVis]).

```

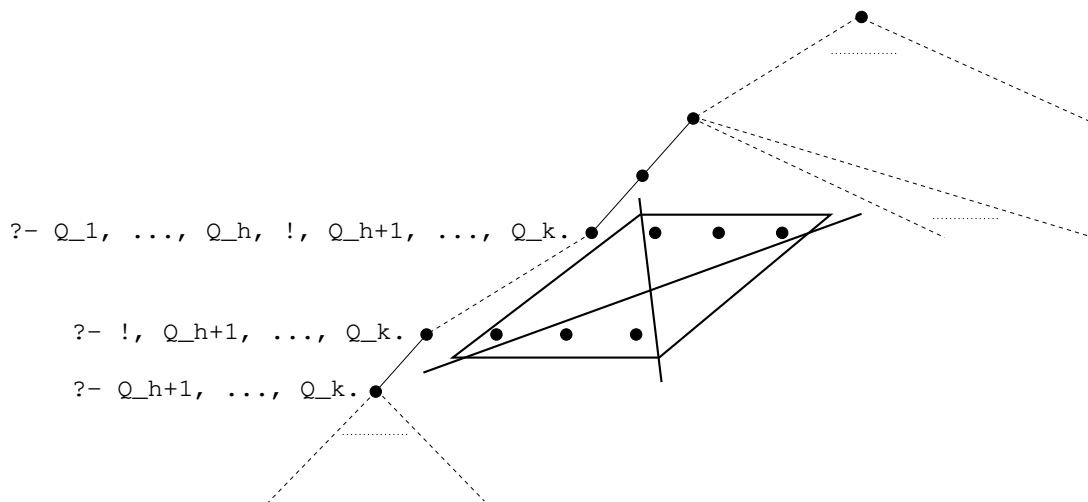


Figura 8.1: Effetto del taglio sull'albero e-o

- Prolog mette a disposizione dei predicati predefiniti per leggere, aggiungere o eliminare clausole di un programma così come per invocare obiettivi, grazie al fatto che le clausole e gli obiettivi stessi sono visti come termini in cui simboli come `:-` e la virgola sono visti come predicati in notazione infissa (in Prolog non c'è distinzione tra istruzioni e dati):
 - `clause(H, B)` ha successo sse il programma contiene una clausola la cui testa è unificabile con `H` e il cui corpo è unificabile con `B` (l'unificazione avviene con la prima clausola siffatta; il corpo è `true` nel caso di un fatto).
 - `asserta(C)` aggiunge la clausola `C` all'inizio del programma.
 - `assertz(C)` aggiunge la clausola `C` alla fine del programma.
 - `retract(C)` ha successo sse il programma contiene una clausola unificabile con `C` e in tal caso elimina dal programma la prima clausola unificabile con `C`.
 - `retractall(H)` ha successo sse il programma contiene una clausola la cui testa è unificabile con `H` e in tal caso elimina dal programma tutte le clausole la cui testa è unificabile con `H`.
 - `call(G)` invoca `G` come obiettivo corrente e ha successo sse `G` ha successo.

Questi predicati sono utilizzabili sia all'interno dei programmi (i quali possono quindi autoispezionarsi e automodificarsi) che al volo dentro gli obiettivi. Nel primo caso, le aggiunte o eliminazioni che essi determinano hanno luogo a tempo di esecuzione nell'interprete in cui i programmi sono stati caricati, non nei sorgenti dei programmi.

- Un metainterprete è un interprete scritto nello stesso linguaggio nel quale sono scritti i programmi da interpretare. Grazie ai predicati predefiniti per accedere alle clausole, è piuttosto semplice scrivere metainterpreti in Prolog:
 - Metainterprete per Prolog puro:


```
raggiungi(true).
raggiungi((G1, G2)) :- raggiungi(G1), raggiungi(G2).
raggiungi(G)         :- clause(G, B), raggiungi(B).
```
 - Metainterprete per Prolog puro esteso con taglio e negazione:


```
raggiungi(true).
raggiungi((G1, G2)) :- raggiungi(G1), raggiungi(G2).
raggiungi(!)        :- !.
raggiungi(not(G))    :- not(raggiungi(G)).
raggiungi(G)         :- clause(G, B), raggiungi(B).
```